

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH. Durga Prasad	Reg. No.:	192425305
Section / Batch:	2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH. Durga Prasad

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				

Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Project: Huffman Coding vs Fixed-Length Coding for Data Compression

1. Project Title

Comparative Analysis of Huffman Coding and Fixed-Length Coding for Efficient Data Compression

2. Domain

Data Compression

This project belongs to the **Data Compression** domain and focuses on comparing two techniques for representing data using binary codes. The project demonstrates how **Huffman Coding**, a Greedy Algorithm, can reduce the number of bits required to store data compared with traditional **Fixed-Length Coding**.

3. Introduction

Data compression is the process of reducing the amount of storage space required to represent data. It is important because large amounts of text, documents, images, audio, and other digital information need to be stored and transmitted efficiently. **Fixed-Length Coding** represents every character using the same number of bits, regardless of how frequently the character occurs. In contrast, **Huffman Coding** is a lossless compression technique that assigns shorter binary codes to frequently occurring characters and longer codes to less frequent characters. The proposed project compares both techniques using the same input data and calculates the number of bits required by each method. The system also calculates the compression ratio and percentage of space saved. This helps demonstrate why frequency-based coding can provide better compression for data with uneven character frequencies.

4. Problem Statement

Traditional fixed-length coding assigns the same number of bits to every character. This can result in unnecessary storage because frequently occurring characters receive the same number of bits as rarely occurring characters.

For example, if a text contains:

- A → 50 times
- B → 25 times
- C → 15 times
- D → 10 times

Fixed-length coding gives all four characters the same **2-bit code**. However, Huffman Coding can assign fewer bits to frequently occurring characters such as **A**.

Therefore, the project aims to develop a system that **compares Fixed-Length Coding and Huffman Coding** and determines which method provides better storage efficiency.

5. Objectives

No. Objective

- 1 Understand the fundamentals of data compression.
- 2 Implement Fixed-Length Coding.
- 3 Implement Huffman Coding using a Greedy approach.
- 4 Calculate character frequencies from the input data.
- 5 Generate Huffman codes based on character frequency.
- 6 Calculate the total bits required by both methods.
- 7 Calculate compression ratio and percentage of space saved.
- 8 Compare the performance of both techniques.
- 9 Demonstrate the results using tables and charts.
- 10 Identify the most suitable technique for efficient data storage.

6. Existing System

In a basic fixed-length encoding system, each character is assigned the same number of bits.

For example, consider four characters:

Character	Binary Code
A	00
B	01
C	10
D	11

Every character requires **2 bits**, irrespective of its frequency.

Problems with Fixed-Length Coding

1. It does not consider character frequency.
2. Frequently occurring characters are not given shorter codes.
3. It may require more storage.
4. It provides limited compression.
5. It is less efficient when character frequencies are highly unequal.

7. Proposed System

The proposed system implements and compares:

Method 1: Fixed-Length Coding

Every character receives the same number of bits.

Method 2: Huffman Coding

Characters are assigned variable-length prefix codes according to their frequencies.

The system accepts an input text, analyzes character frequencies, generates codes using both approaches, calculates storage requirements, and displays a comparison.

Basic Flow

Input Text → Frequency Analysis → Encoding → Bit Calculation → Compression Comparison → Result

8. Working of the Project

The project can be divided into the following stages:

Stage	Process
1	User enters text
2	System identifies unique characters
3	Frequency of each character is calculated
4	Fixed-length codes are generated
5	Huffman tree is constructed
6	Huffman codes are generated
7	Input text is encoded using both methods
8	Total bits are calculated
9	Compression ratio is calculated
10	Results are displayed and compared

9. Fixed-Length Coding

Fixed-length coding assigns an identical number of bits to every symbol.

If there are **N unique characters**, the number of bits required for each character is:

$$L = \lceil \log_2(N) \rceil$$

For example, if there are four unique characters:

$$L = \log_2(4) = 2$$

Therefore, each character requires 2 bits.

Example

Character	Frequency	Fixed Code	Bits per Character	Total Bits
A	50	00	2	100
B	25	01	2	50
C	15	10	2	30
D	10	11	2	20
Total	100			200

10. Huffman Coding

Huffman Coding is a **lossless data compression technique** based on a Greedy Algorithm.

The basic idea is to give:

- **Short codes** → **frequently occurring characters**
- **Long codes** → **less frequently occurring characters**

For the same example:

Character	Frequency	Huffman Code	Code Length
A	50	0	1
B	25	10	2
C	15	110	3
D	10	111	3

The total number of bits becomes:

$$\begin{aligned} & (50 \times 1) + (25 \times 2) + (15 \times 3) + (10 \times 3) \\ &= 50 + 50 + 45 + 30 \\ &= 175 \text{ bits} \end{aligned}$$

11. Huffman Tree Construction

Huffman Coding creates a binary tree using character frequencies.

Given:

Character	Frequency
A	50
B	25
C	15
D	10

First, select the two characters with the smallest frequencies:

$$10 + 15 = 25$$

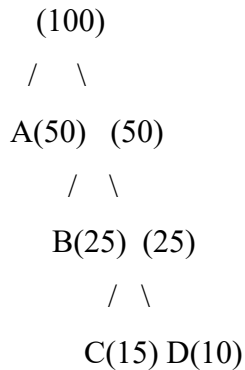
Then combine:

$$25 + 25 = 50$$

Finally:

$$50 + 50 = 100$$

A possible tree is:



The resulting codes can be:

Character	Huffman Code
A	0
B	10
C	110
D	111

The exact binary values can differ depending on how ties are handled, but the code lengths and compression efficiency remain optimal.

12. Comparison of Both Methods

Feature	Fixed-Length Coding	Huffman Coding
Code type	Fixed	Variable
Character frequency	Not considered	Considered
Frequent characters	Same code length	Shorter codes
Rare characters	Same code length	Longer codes
Compression	Low	High
Algorithm	Direct encoding	Greedy Algorithm
Tree required	No	Yes
Implementation	Easy	Moderate
Storage efficiency	Lower	Higher
Best for unequal frequencies	No	Yes
Prefix-free	Can be designed as prefix-free	Yes
Main advantage	Simplicity	Compression efficiency

13. Performance Analysis

Using the example of 100 characters:

Parameter	Fixed-Length	Huffman
Number of characters	100	100
Bits required	200	175
Bits saved	—	25
Percentage reduction	0%	12.5%
Compression efficiency	Lower	Higher

Percentage reduction:

$$\frac{200 - 175}{200} \times 100 \\ = 12.5\%$$

Therefore, Huffman Coding reduces the required number of bits by **12.5%** for this example.

14. Compression Ratio

The compression ratio can be calculated as:

$$\text{Compression Ratio} = \frac{\text{Original Size}}{\text{Compressed Size}}$$

For comparison between the two coding methods:

$$\frac{200}{175} = 1.14$$

Therefore, the Huffman representation is approximately **1.14 times more compact** than the fixed-length representation for this example.

15. Algorithm

Huffman Coding Algorithm

1. Read the input text.
2. Count the frequency of every character.
3. Create a node for every character.
4. Insert all nodes into a priority queue.
5. Select the two nodes with the smallest frequencies.
6. Combine them into a new node.
7. Insert the new node back into the priority queue.
8. Repeat until only one node remains.
9. The remaining node becomes the root of the Huffman Tree.

10. Assign 0 to the left branch and 1 to the right branch.
11. Generate the Huffman code for each character.
12. Encode the input text.
13. Calculate the total number of bits.
14. Compare with Fixed-Length Coding.
15. Display compression results.

16. Pseudocode

START

Read input text

Find frequency of each character

Calculate Fixed-Length bits

Create Huffman nodes using character frequencies

Insert nodes into priority queue

WHILE more than one node exists:

 Remove two nodes with minimum frequency

 Create a new parent node

 Add frequencies

 Insert parent node back

Generate Huffman codes from the tree

Encode input text using Huffman codes

Calculate Huffman bits

Compare Fixed-Length bits and Huffman bits

Calculate:

 Bits Saved

 Compression Ratio

 Percentage Reduction

Display results

END

17. Sample Input and Output

Input

AAAABBBCCD

Frequency table:

Character	Frequency
-----------	-----------

A	4
B	3
C	2
D	1

Fixed-Length Coding

There are 4 unique characters, so:

$$\log_2(4) = 2$$

Total characters:

$$4 + 3 + 2 + 1 = 10$$

Total bits:

$$10 \times 2 = 20$$

Huffman Coding

A possible Huffman code is:

Character	Frequency	Huffman Code
A	4	0
B	3	10
C	2	110
D	1	111

Total:

$$\begin{aligned}
 &(4 \times 1) + (3 \times 2) + (2 \times 3) + (1 \times 3) \\
 &= 4 + 6 + 6 + 3 \\
 &= 19 \text{ bits}
 \end{aligned}$$

Final Comparison

Parameter	Fixed-Length	Huffman
Total characters	10	10
Total bits	20	19
Bits saved	—	1
Percentage reduction	—	5%

18. Technologies Required

Component	Suggested Technology
Programming Language	Python
Algorithm	Huffman Greedy Algorithm
Data Structure	Priority Queue / Min Heap

Data Storage	Lists, Dictionaries
Visualization	Matplotlib
Development Environment	VS Code / Jupyter Notebook
Input	Text file or user-entered text
Output	Tables, codes, compression statistics

For a simple academic project, **Python + Min Heap + Matplotlib** is a good combination.

19. Main Modules

Module 1 – Input Data Management

The user provides text or uploads a text file. The system validates and processes the input.

Module 2 – Frequency Analysis

The system counts how many times each character occurs.

Module 3 – Fixed-Length Encoding

The system determines the number of bits required and assigns equal-length binary codes.

Module 4 – Huffman Tree Generation

The system constructs the Huffman Tree using a **Min Heap/Priority Queue**.

Module 5 – Huffman Encoding

The system traverses the tree and generates a unique prefix-free code for each character.

Module 6 – Performance Comparison

The system compares the number of bits required by Fixed-Length and Huffman Coding.

Module 7 – Result Visualization

The system displays tables and graphs showing compression performance.

20. Time Complexity

Let:

- **n** = number of input characters
- **k** = number of unique characters

Frequency calculation:

$$O(n)$$

Building the Huffman Tree:

$$O(k \log k)$$

Generating codes:

$$O(k)$$

Encoding the text:

$$O(n)$$

Therefore, the overall complexity is approximately:

$$O(n + k \log k)$$

Since $k \leq n$, this is commonly expressed as:

$$O(n \log n)$$

for general analysis.

21. Space Complexity

The system requires memory for the frequency table, Huffman tree, codes, and encoded data.

Component	Space Complexity
Input text	$O(n)$
Frequency table	$O(k)$
Huffman tree	$O(k)$
Code table	$O(k)$
Encoded output	$O(n)$
Overall	$O(n + k)$

22. Advantages of the Project

1. Demonstrates an important application of **Greedy Algorithms**.
2. Provides a practical understanding of data compression.
3. Shows the difference between fixed and variable-length coding.
4. Reduces storage requirements for suitable data.
5. Uses lossless compression, so the original data can be recovered.
6. Provides measurable performance results.
7. Can be implemented using simple data structures.
8. Can be extended to text-file compression.

23. Limitations

Limitation	Explanation
Frequency dependent	Huffman performs better when symbol frequencies are unequal.
Tree overhead	The Huffman tree/code table must also be stored or transmitted.
Implementation complexity	More complex than fixed-length coding.
Small files	Compression benefits may be small because of tree overhead.

Not universal	Huffman is not always the best compression method for every type of data.
---------------	---

24. Future Enhancements

The project can be extended by adding:

- **Text file compression and decompression**
- Graphical User Interface
- File upload/download functionality
- Compression of multiple file formats
- Compression ratio graphs
- Comparison with **Run-Length Encoding**
- Comparison with **Arithmetic Coding**
- Comparison with **LZW**
- Real-time compression statistics
- Visualization of the Huffman Tree

25. Expected Result

The expected result is that **Huffman Coding requires fewer bits than Fixed-Length Coding when the input contains characters with different frequencies.**

For example:

Method	Bits Required
Fixed-Length Coding	200
Huffman Coding	175
Bits Saved	25
Space Reduction	12.5%

Thus, the project demonstrates that frequency-based Huffman Coding can improve storage efficiency while maintaining **lossless data recovery**.

26. Conclusion

The project “**Comparative Analysis of Huffman Coding and Fixed-Length Coding for Efficient Data Compression**” demonstrates how different coding strategies affect storage requirements. Fixed-Length Coding is simple because every character uses the same number of bits, but it does not take character frequency into account. Huffman Coding overcomes this limitation by using a **Greedy Algorithm** to assign shorter codes to frequently occurring characters and longer codes to less frequent characters. The project calculates the total bits, compression ratio, percentage

reduction, and storage savings for both techniques. The results generally show that Huffman Coding provides better compression when character frequencies are uneven. Therefore, the project provides a practical demonstration of **Greedy Algorithm optimization in the Data Compression domain**.